

# SQL ORACLE

## ACTUALIZACIÓN DE DATOS

Forma parte del DML (**Data Manipulation Language**). Al conjunto de instrucciones DML que se ejecutan consecutivamente, se las llama **transacciones** y se pueden anular todas ellas o aceptar, ya que una instrucción DML no es realmente efectuada hasta que no se acepta con una instrucción **COMMIT**.

En todas estas consultas, el único dato devuelto por Oracle es el número de registros que se han modificado.

## INSERCIÓN de DATOS (INSERT)

La adición de datos a una tabla se realiza mediante la instrucción **INSERT**.

Su sintaxis fundamental es:

```
INSERT INTO tabla [(listaDeCampos)]  
VALUES (valor1 [,valor2 ...])
```

La **tabla** representa la tabla a la que queremos añadir el registro y los valores que siguen a **VALUES** son los valores que damos a los distintos campos del registro. Si no se especifica la lista de campos, la lista de valores debe seguir el orden de las columnas según fueron creados (es el orden de columnas según las devuelve el comando **DESCRIBE** o **DESC**).

La lista de campos a rellenar se indica si no queremos rellenar todos los campos. Los campos no rellenados explícitamente con la orden **INSERT**, se rellenan con su valor por defecto (**DEFAULT**) o bien con **NULL** si no se indicó valor alguno. Si algún campo tiene restricción de obligatoriedad (**NOT NULL**), ocurrirá un error si no rellenamos el campo con algún valor. Por ejemplo, supongamos que tenemos una tabla de alumnos cuyos campos son: **dni, nombre, apellido1, apellido2, localidad y dirección**; supongamos que ese es el orden de creación de los campos de esa tabla y que la localidad tiene como valor por defecto **Bilbao** y la dirección no tiene valor por defecto. En ese caso estas dos instrucciones son equivalentes:

```
INSERT INTO clientes VALUES( '11111111',Laura,'Parlón', 'Arias',DEFAULT,NULL);
```

```
INSERT INTO clientes(dni,nombre,apellido1,apellido2)  
VALUES('11111111',Laura,'Parlón', 'Arias');
```

Son equivalentes, puesto que en la segunda instrucción los campos no indicados se rellenan con su valor por defecto y la dirección no tiene valor por defecto. La palabra **DEFAULT** fuerza a utilizar ese valor por defecto.

## INSERCIÓN con SELECT (INSERT INTO .....SELECT)

Podemos insertar registros en una tabla tomando los datos de otra tabla. La sintaxis es la siguiente:

```
INSERT INTO nombretabla [(columna [,columns]...]  
    SELECT {columna [,columna]... | *}  
    FROM nombretabla2 [cláusulas de SELECT];
```

Tenemos una tabla denominada P\_EMPLEADOS con la misma descripción de campos de EMPLEADOS. Insertar en la tabla P\_EMPLEADOS todos los empleados de la oficina 12.

```
INSERT INTO P_EMPLEADOS SELECT * FROM EMPLEADOS WHERE idoficina=12;
```

```
INSERT INTO P_EMPLEADOS (idempleado, nombre, fnacimiento, fcontrato,...) SELECT  
(idempleado, nombre, fnacimiento, fcontrato,...) FROM EMPLEADOS WHERE idoficina=12;
```

Insertar un empleado JOSE GARCIA con numero de empleado 888, en la oficina que más empleados haya. La fecha de contrato será la del día de hoy.

```
INSERT INTO P_EMPLEADOS(idempleado,nombre,fnacimiento,fcontrato,idoficina)  
VALUES (888,'JOSE GARCIA', '10/10/1972', sysdate , (SELECT oficina FROM EMPLEADOS  
GROUP BY idoficina HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM  
EMPLEADOS GROUP BY idoficina));
```

## ACTUALIZACIÓN de DATOS (UPDATE)

La modificación de los datos de los registros lo implementa la instrucción **UPDATE**.

Sintaxis:

```
UPDATE tabla  
SET columna1=valor1 [,columna2=valor2...]  
[WHERE condición]
```

Se modifican las columnas indicadas en el apartado SET con los valores indicados. La cláusula WHERE permite especificar qué registros serán modificados.

Ejemplos:

```
UPDATE alumnos SET curso='DAW'  
WHERE curso='DAI';
```

```
UPDATE productos SET punitario=punitario*1.16;
```

El primer update actualiza el curso de los alumnos. A todos los alumnos que tengan como curso DAI, les va a poner a partir de ahora DAW.

El segundo UPDATE incrementa los precios de todos los productos en un 16%.

La expresión para el valor puede ser todo lo compleja que se desee (en el ejemplo se utilizan funciones de fecha para conseguir que los pedidos que se deben entregar hoy se entreguen el miércoles que viene).

```
UPDATE pedidos SET fechaentrega= NEXT_DAY(SYSDATE,'Miércoles')
WHERE fecha=SYSDATE;
```

## BORRADO de DATOS (DELETE)

Se realiza mediante la instrucción DELETE.

```
DELETE [FROM] tabla
[WHERE condición]
```

Es más sencilla que las anteriores, elimina los registros de la tabla que cumplan la condición indicada. Ejemplo:

```
DELETE FROM empleados
WHERE idoficina=11;
```

Hay que tener en cuenta que el borrado de un registro no puede provocar fallos de integridad y que la opción de **integridad ON DELETE CASCADE** hace que no sólo se borren los registros indicados en el SELECT, sino todos los relacionados.

## TRABAJO CON TRANSACCIONES (COMMIT, ROLLBACK)

Una transacción está formada por una serie de instrucciones DML. Una transacción comienza con la primera instrucción DML que se ejecute y finaliza con alguna de estas circunstancias:

Una operación **COMMIT** o **ROLLBACK**

Una instrucción DDL (como **ALTER TABLE** por ejemplo)

Una instrucción DCL (como **GRANT**)

El usuario abandona la sesión

Caída del sistema

Hay que tener en cuenta que cualquier instrucción DDL o DCL da lugar a un COMMIT implícito, es decir todas las instrucciones DML ejecutadas hasta ese instante pasan a ser definitivas.

## COMMIT

La instrucción COMMIT hace que los cambios realizados por la transacción sean definitivos, irrevocables. Sólo se debe utilizar si estamos de acuerdo con los cambios, conviene asegurarse mucho antes de realizar el COMMIT ya que las instrucciones ejecutadas pueden afectar a miles de registros.

Además, el cierre correcto de la sesión da lugar a un COMMIT, aunque siempre conviene ejecutar explícitamente esta instrucción a fin de asegurarnos de lo que hacemos.

## ROLLBACK

Esta instrucción regresa a la instrucción anterior al inicio de la transacción, normalmente el último COMMIT, la última instrucción DDL o DCL o al inicio de sesión. Anula definitivamente los cambios, por lo que conviene también asegurarse de esta operación.

Un abandono de sesión incorrecto o un problema de comunicación o de caída del sistema dan lugar a un ROLLBACK implícito.

Estado de los datos durante la transacción

Si se inicia una transacción usando comandos DML hay que tener en cuenta que:

Se puede volver a la instrucción anterior a la transacción cuando se desee

Las instrucciones de consulta SELECT realizadas por el usuario que inició la transacción muestran los datos ya modificados por las instrucciones DML

El resto de usuarios ven los datos tal cual estaban antes de la transacción, de hecho los registros afectados por la transacción aparecen bloqueados hasta que la transacción finalice. Esos usuarios no podrán modificar los valores de dichos registros.

Tras la transacción todos los usuarios ven los datos tal cual quedan tras el fin de transacción.

Los bloqueos son liberados y los puntos de ruptura borrados.

## SAVEPOINT

Se pueden colocar puntos de Salvaguarda para realizar rollbacks hasta ese punto determinado. El punto de salvaguarda se colocará con la instrucción:

SAVEPOINT A;

Y se desharán todas las instrucciones que se hayan hecho posteriormente al punto de salvaguarda A ejecutando la instrucción:

ROLLBACK TO A;